

□□ IAP □□ Original EGO Planner □□ Audit

□□□□□□□□□□ IAP □□□□ EGO planner □□□□□□□□□□ Safety Planner □□□□□□□□□□ planner □□□□:

```
src/iap/src/iap/planner/...
```

```
src/iap/sim/ego_planner_swarm_ws/...
```

1. Audit ☐☐

- EGO planner occupancy map rebound B-spline planner GridMap FSM
- EGOPlanerManager::reboundReplan() BsplineOptimizer swarm clearance B-spline

■■■■■■■■■■ IAP ■ integrity risk■■■ B splineOptimizer::combineCostRebound() ■■■:

smoothness + collision distance + feasibility + swarm + terminal

```
AStar::AstarSearch()  occupied voxel  risk cost  FSM  50 ms safety callback  depth timeout  swarm
PL_pred < AL
```

□□ Safety Planner □□□□□□□□□□ planner□□□□□□□□□□:

- P5: runtime integrity supervisor
- P2: distinctive trajectory candidate ranking
- P1: backend soft integrity cost
- P3: global/local reference bias
- P4: risk-aware A* as needed

demo9 / original planner

2. □□□□□□□□□□

	模块	实现文件	主要功能	完整性风险
PROF	planner node	ego_planner_node.cpp	调用 EGOReplanFSM 的 spin()	低
	FSM	EGOReplanFSM	负责安全检查和规划逻辑	低
	planner manager	EGOPlannerManager	管理全局/局部规划	低
	map	GridMap	管理 odometry/cloud/depth 和 occupancy 图	低
	A*	AStar::AstarSearch()	负责寻路	低
	optimizer	BsplineOptimizer	B-spline 优化器，使用 L-BFGS	低
	traj server	traj_server.cpp	提供 B-spline 规划命令	低

- ROS topic / remap
- advanced param.launch.py:

```
odom_world      <- odometry_topic
grid_map/odom   <- odometry_topic
grid_map/cloud  <- cloud_topic
grid_map/depth  <- depth_topic
planning/bspline -> drone <id> planning/bspline
```

3. Original Planner Mermaid

original EGO planner [P*] Safety Planner

```
flowchart TD
    %% =====
    %% Inputs
    %% =====
    I0["odom_world<br/>nav_msgs/Odometry"] --> F0
    I1["grid_map/odom<br/>nav_msgs/Odometry"] --> M0
    I2["grid_map/cloud & grid_map/depth<br/>PointCloud2 / Image"] --> M0
    I3["goal / trigger<br/>move_base_simple/goal & /traj_start_trigger"] --> F1
    I4["swarm trajectories<br/>MultiBsplines / Bspline"] --> F2
    I5["P0 <br/>Predictor / RiskGrid / integrity cost field<br/>planner P5"] --> S0
    S0 --> S0

    %% =====
    %% Initialization
    %% =====
    N0["ego_planner_node<br/> EGOReplanFSM"] --> F0["EGOReplanFSM::init()"]
    F0 --> PM0["EGOPlanerManager::initPlanModules()"]
    PM0 --> M0["GridMap::initMap(<br/> odom/cloud/depth<br/> occupancy_buffer & occupancy_buffer_inflate"]
    PM0 --> O0["BsplineOptimizer::setParam(<br/> lambda_smooth / collision / feasibility / fitness"]
    PM0 --> A0["AStar::initGridMap(grid_map)"]
    F0 --> T0["timer<br/>10 ms execFSMCallback(<br/>50 ms checkCollisionCallback()"]

    %% =====
    %% Map update
    %% =====
    M0 --> M1["GridMap callbacks<br/>cloudCallback / depthOdomCallback / odomCallback"]
    M1 --> M2["updateOccupancyCallback(<br/>raycast + log-odds update"]
    M2 --> M3["clearAndInflateLocalMap(<br/> inflated occupancy"]
    M3 --> Q0["query: getInflateOccupancy(pos)<br/> A* / optimizer / safety check"]

    %% =====
    %% FSM target handling
    %% =====
    F1 --> F3["waypointCallback() / triggerCallback()"]
    F3 --> G0["planNextWaypoint()"]
    G0 --> G1["EGOPlanerManager::planGlobalTraj(<br/> global min-snap reference"]
    G1 --> G2["GlobalTrajData::setGlobalTraj()"]
    G2 --> F4["FSM: WAIT_TARGET -> GEN_NEW_TRAJ<br/> EXEC_TRAJ -> REPLAN_TRAJ"]

    F2 --> SW0["swarmTrajsCallback() / BroadcastBsplineCallback(<br/>"]
    SW0 --> SW1["EGOPlanerManager::checkCollision(id)<br/> swarm"]
    SW1 --> F_REPLAN["FSM -> REPLAN_TRAJ"]

    %% =====
    %% FSM execution
    %% =====
    T0 --> F5["execFSMCallback() 10 ms"]
    F5 --> FS0["FSM state"]
    FS0 --> INIT["INIT"]
    FS0 --> WAIT_TARGET["WAIT_TARGET"]
    FS0 --> SEQUENTIAL_START["SEQUENTIAL_START / GEN_NEW_TRAJ"]
    FS0 --> REPLAN_TRAJ["REPLAN_TRAJ"]
    FS0 --> EXEC_TRAJ["EXEC_TRAJ"]
    FS0 --> EMERGENCY_STOP["EMERGENCY_STOP"]

    EX0 --> F_REPLAN
    EX0 --> EXEC_OK["EXEC_TRAJ"]
    EM0 --> PUB_STOP["stop B-spline"]

    %% =====
```

```

%% Local planning
%% =====
PSTART --> C0["callReboundReplan(use_poly_init=true/false)"]
PCUR --> C0
C0 --> LT0["getLocalTarget()<br/>[] global reference [] local target"]
LT0 --> P3["[P3 []]<br/>global reference [] local target selection<br/>[]"]
LT0 --> RR0["EGOPlannerManager::reboundReplan()"]

RR0 --> R1["STEP 1 INIT<br/>[] point_set"]
R1 --> R1A["[]"]
R1A -->|"first / poly init"| R2["PolynomialTraj min-snap<br/>start -> local target"]
R1A -->|"replan from current traj"| R3["[] B-spline []<br/>[] local target"]
R2 --> R4["UniformBspline::parameterizeToBspline()<br/>point_set -> ctrl_pts"]
R3 --> R4

R4 --> IC0["BsplineOptimizer::initControlPoints(ctrl_pts)<br/>[] inflated occupancy []"]
IC0 --> IC1["[]?"]
IC1 -->|"[]"| OPT0["[] rebound optimize"]
IC1 -->|"[]"| AST0["[] AStar::AstarSearch()<br/>[]"]
AST0 --> P4["[P4 []]<br/>A* edge cost [] risk-aware cost<br/>[] P1/P2 []"]
AST0 --> IC2["[] A* path [] base_point / direction<br/>[] rebound distance cost []"]
IC2 --> OPT0

%% =====
%% Optimization
%% =====
OPT0 --> D0{"manager/use_distinctive_trajs?"}
D0 -->|"false"| B00["BsplineOptimizeTrajRebound(ctrl_pts, ts)"]
D0 -->|"true"| DT0["BsplineOptimizer::distinctiveTrajs(segments)<br/>[] control points"]
DT0 --> DT1["[] BsplineOptimizeTrajRebound()<br/>[] final_cost []"]
DT1 --> P2["[P2 []]<br/>[] integrity score []<br/>[]"]
DT1 --> B01["[] winning ctrl_pts"]
B00 --> B01

B01 --> LB0["rebound_optimize()<br/>L-BFGS []"]
LB0 --> CC0["combineCostRebound()"]
CC0 --> CC1["[] cost:<br/>smoothness + collision distance + feasibility + swarm + terminal"]
CC1 --> P1["[P1 []]<br/>[] integrity soft cost<br/>[] PL_pred < AL []"]
CC1 --> LB1["[] inflated occupancy<br/>[] restart / [] collision lambda"]
LB1 --> RF0["STEP 3 REFINE<br/>[] reallocate time<br/>check velocity/acc feasibility"]
RF0 --> UP0["updateTrajInfo()<br/>[] local_data_ :
position/velocity/acceleration/duration/start_time/traj_id"]

%% =====
%% Publishing and execution
%% =====
UP0 --> PUB0["EGOReplanFSM::callReboundReplan()<br/>[] traj_utils/Bspline"]
PUB0 --> PUB1["[] planning/bspline"]
PUB0 --> PUB2["publishSwarmTrajs()<br/>[] swarm / broadcast bspline"]
PUB1 --> TS0["traj_server::bsplineCallback()<br/>[] B-spline [] knots"]
TS0 --> TS1["traj_server::cmdCallback()<br/>[] pos/vel/acc/yaw"]
TS1 --> OUT0["[] quadrotor_msgs/PositionCommand<br/>[]/[]"]
PUB1 --> VIS0["PlanningVisualization<br/>[] init/optimal/global path"]

%% =====
%% Original safety loop and P5
%% =====
T0 --> SAFE0["checkCollisionCallback() 50 ms<br/>original safety check"]
SAFE0 --> SDEPTH["GridMap depth/odom timeout?"]
SDEPTH -->|"[]"| SEMG["FSM -> EMERGENCY_STOP"]
SDEPTH -->|"[]"| STRAJ["[] local_data_ B-spline []<br/>[] 2/3"]
STRAJ --> SOCC["getInflateOccupancy(pos) [] swarm distance unsafe?"]
SOCC -->|"[]"| SAFE_OK["[]"]
SOCC -->|"[]"| SCHANCE["[] planFromCurrentTraj()"]
SCHANCE -->|"[]"| EXEC_OK
SCHANCE -->|"[]"| SCHANCE["[] < emergency_time"| SEMG
SCHANCE -->|"[]"| F_REPLAN

SAFE0 --> S0["[P5 []]<br/>[] checkTrajIntegrity()<br/>iap/integrity + Predictor +
AlertLimitProvider"]

```

```

S0 --> S1["state: ARAIM monitor <br/>HPL/VPL/HAL/VAL/IM/state"]
S0 --> S2["state: local_data_ B-spline <br/>Predictor PL_pred<br/>AlertLimitProvider AL"]
S1 --> S3{"PL / AL / IM unsafe unknown?"}
S2 --> S3
S3 -->|"safe"| SAFE_OK
S3 -->|"future violation, not imminent"| F_REPLAN
S3 -->|"current unsafe imminent"| SEMG

```

4. 4.1

4.1

`GridMap::initMap()` planner node `grid_map/odom``grid_map/cloud``grid_map/depth` raycast
`log-odds` `occupancy_buffer` `clearAndInflateLocalMap()` `occupancy_buffer_inflate_`

`A*B-spline optimizer` `safety callback`:

```
grid_map_->getInflateOccupancy(pos)
```

obstacle occupancy GNSS/LiDAR integrity risk

4.2 FSM

`EG0ReplanFSM::init()` timer:

```

10 ms: execFSMCallback()
50 ms: checkCollisionCallback()

```

`execFSMCallback()`:

```

INIT -> WAIT_TARGET -> SEQUENTIAL_START / GEN_NEW_TRAJ -> EXEC_TRAJ
EXEC_TRAJ -> REPLAN_TRAJ
safety unsafe -> EMERGENCY_STOP

```

manual target `/move_base_simple/goal` preset target launch waypoints `/traj_start_trigger`

FSM

FSM

PROF

odom

odom WAIT_TARGET

WAIT_TARGET trigger target/trigger SEQUENTIAL_START

SEQUENTIAL_START swarm planFromGlobalTraj()

GEN_NEW_TRAJ global reference EXEC_TRAJ

REPLAN_TRAJ EXEC_TRAJ

EXEC_TRAJ B-spline

EMERGENCY_STOP stop trajectory fail-safe GEN_NEW_TRAJ

4.3 Global Reference Local Target

`planNextWaypoint()`:

```
EG0PlannerManager::planGlobalTraj(...)
```

min-snap polynomial global reference `global_data` FSM `callReboundReplan()`
`getLocalTarget()` global reference planning horizon local target

obstacle-aware search global reference integrity risk

4.4 Local Replan

`planFromGlobalTraj()` → `planFromCurrentTraj()` 流程图:

```
EG0ReplanFSM::callReboundReplan()  
-> EG0PlannerManager::reboundReplan()
```

`reboundReplan()` 流程:

1. INIT: 初始化 parameterize 的 B-spline control points
2. OPTIMIZE: 调用 rebound optimizer
3. REFINE: 迭代优化/细化

流程图:

```
first / force poly init:  
start -> local target 的 polynomial trajectory  
  
replan from current traj:  
调用 B-spline 的 + local target
```

4.5 A* 搜索

`BsplineOptimizer::initControlPoints()` 初始化 inflated occupancy 碰撞 segments

碰撞 segments 的 segment 列表:

```
AStar::AstarSearch(step_size, in, out)
```

A* 搜索 EGO 的 guide path 的 `base_point` 和 `direction` 距离 cost 的 B-spline

的 P4 risk-aware A* 搜索 A* 搜索

4.6 B-spline Optimization

`BsplineOptimizer::rebound_optimize()` 调用 L-BFGS 优化:

```
BsplineOptimizer::combineCostRebound()
```

流程图:

```
lambda_smooth      * smoothness  
new_lambda_collision * collision distance  
lambda_feasibility * feasibility  
new_lambda_collision * swarm  
lambda_collision    * terminal
```

的 integrity cost 的 `PL_pred < AL`

`manager/use_distinctive_trajs=true` 调用 `distinctiveTrajs()` 计算 `final_cost` 的 P2 candidate ranking

4.7 总结

调用 `callReboundReplan()` 的 `local_data_.position_traj_`:

traj_utils/msg/Bspline

~~~~~:

planning/bspline

traj\_server 订阅 topic 订阅 bsplineCallback() 订阅 knots 订阅 cmdCallback() 订阅 position 订阅 velocity 订阅 acceleration 订阅 yaw 订阅 quadrotor\_msgs/PositionCommand 订阅

4.8 安全 Safety Check

checkCollisionCallback() 每 50 ms 检查一次:

1. 检查 depth/odom timeout 是否 EMERGENCY\_STOP
2. 检查 t\_cur 是否 B-spline 结束
3. 检查 inflated occupancy
4. 检查
5. 检查 unsafe 是否 planFromCurrentTraj()
6. 检查 emergency\_time\_ 是否 EMERGENCY\_STOP 是否 REPLAN\_TRAJ

安全 safety loop 在 P5 检查碰撞 P5 检查 collision check 检查 occupancy safety

5. Safety Planner 安全规划

| 名称 | 描述                           | 安全等级                                        | 安全策略                                                      | 安全策略  |
|----|------------------------------|---------------------------------------------|-----------------------------------------------------------|-------|
| P0 | 安全                           | Predictor / RiskGrid / integrity cost field | 安全 planner 安全 EGO 安全                                      | 安全    |
| P5 | Runtime Integrity Supervisor | EG0ReplanFSM::checkCollisionCallback() 安全   | 安全 checkTrajIntegrity() 安全 /iap/integrity 安全 PL_pred < AL | 安全    |
| P2 | 安全                           | manager/use_distinctive_trajs 安全            | 安全 integrity score 安全                                     | 安全    |
| P1 | 安全 soft cost                 | BsplineOptimizer::combineCostRebound() 安全   | 安全 calcIntegrityCost() 安全 cost 安全                         | 安全    |
| P3 | 安全/安全                        | planGlobalTraj() / getLocalTarget() 安全      | 安全 global reference 安全 local target 安全                    | 安全    |
| P4 | risk-aware A*                | AStar::AstarSearch() 安全                     | 安全 risk cost 安全                                           | A* 安全 |

~~~~~:

- P5 检查 PL_pred < AL 安全
- P1/P2/P3/P4 安全 P5 安全 hard safety statement
- /iap/integrity_cost_field 安全 P1 soft cost 安全 P5 安全
- ARAIM 安全 monitor 安全 Predictor future PL 安全: 安全 certified monitor 安全 /iap/integrity 安全 advisory PL 安全 Predictor

6. Audit 安全审计

安全 original planner 安全:

occupancy map -> initial B-spline -> rebound optimize -> publish B-spline -> runtime collision check

Safety Planner 安全规划:

1. **P5**: 安全 FSM safety loop 安全 replan / emergency 安全 optimizer

2. **P2**: `use_distinctive_trajs` integrity ranking
3. **P1**: `combineCostRebound()` integrity soft cost collision feasibility
4. **P3**: global reference local target
5. **P4**: `P1/P2` A^* risk-aware edge cost

baseline:

integrity planner demo9 / original planner

7.

:

src/iap/docs/dev_planner/iap_original_ego_planner_flow_audit.md
Mermaid fence
flowchart TD
P5 / P2 / P1 / P3 / P4

8. P0-P5

P0-P5 original EGO planner

:

- P5 hard safety gate $PL_{pred} < AL$
- P1/P2/P3/P4
- P0 planner original EGO planner
- original EGO planner

8.1 P0: RiskGridMap

P0 RiskGridMap Safety Planner risk lookup table B-spline FSM
P1/P2/P3/P4/P5

P0 original EGO planner P0 10 P0 RiskGridMap EGO GridMap

:

- EGO planner Predictor
- P1/P2/P3/P4 c_{pi} / $grad_{c_{pi}}$
- P5 raw predicted HPL/VPL valid/stale c_{pi} $PL_{pred} < AL$
- RiskGridMap buffer EGO GridMap world/map frame indexing EGO occupancy buffer

original EGO

PROF

| original | P0 |
|---|---|
| <code>EGOPlannerManager::initPlanModules()</code> | <code>RiskGridMap</code> adapter |
| <code>BsplineOptimizer::setEnvironment()</code> | optimizer <code>RiskCostAdapter</code> $c_{pi}/grad_{c_{pi}}$ |
| <code>checkCollisionCallback()</code> | P5 <code>PredictedPLAdapter</code> raw predicted PL / valid / stale |
| <code>distinctiveTrajs()</code> | P2 <code>RiskGridMap</code> |
| <code>combineCostRebound()</code> | P1 <code>getRiskCost()</code> / <code>getRiskGrad()</code> |

```

    PredictorModule
    EGO odom / UAV pose
    EGO GridMap frame / local window / occupancy
    RiskGridMap : resolution, size, refresh period, stale timeout, unknown policy
    planner query position / query time

```

```

RiskQueryResult:
    position
    c_pi
    grad_c_pi
    hpl_pred, vpl_pred
    valid, stale, unknown
    age_s
    reason

```

```

struct RiskQueryResult {
    bool ok;
    bool valid;
    bool stale;
    bool unknown;
    Eigen::Vector3d position;
    double c_pi;
    Eigen::Vector3d grad_c_pi;
    double hpl_pred;
    double vpl_pred;
    double age_s;
    std::string reason;
};

class RiskGridMap {
public:
    bool queryRisk(const Eigen::Vector3d& p_w,
                  double now_s,
                  RiskQueryResult* out) const;
    bool getRiskCost(const Eigen::Vector3d& p_w,
                    double now_s,
                    double* cost) const;
    bool getRiskGrad(const Eigen::Vector3d& p_w,
                    double now_s,
                    Eigen::Vector3d* grad) const;
};

```

Algorithm P0: RiskGridMap query

Input:

| | |
|-----------|-----------------------------------|
| p | query position in map/world frame |
| now_s | query time |
| risk_grid | independent rolling RiskGridMap |
| policy | stale timeout, unknown handling |

Output:

| | |
|---------|--|
| result | RiskQueryResult |
| success | true if result can be used by the caller |

Procedure:


```
1: if p is outside RiskGridMap then
2:   return false with reason = "out_of_range"
3: end if
4: gather 8 neighbor risk voxels
5: if required voxels are invalid or stale then
6:   return false with reason = "stale_or_unknown"
7: end if
8: c_pi <- trilinear interpolation of voxel.c_pi
9: grad_c_pi <- analytic gradient of trilinear interpolation
10: hpl_pred/vpl_pred <- interpolated raw PL if requested
11: return valid result
```

□□□□□□□□

- P0 □□□ risk data □□ EGO GridMap□GridMap □□□□□ occupancy□
- P0 □□□□ FuturePLFieldPredictor□PLGrid□UnifiedRiskGrid□PICostAdapter□alert_limit_model □ planner-side □□
- P0 □□□□□□□□□□ 0 risk□□□□□□□□□□ unknown□
- P1/P2/P3/P4 □□ c_pi/grad_c_pi□P5 □□□ raw predicted PL / valid / stale□□□□ c_pi □ hard safety□

8.2 P5 Runtime Integrity Supervisor: hard safety gate

□□□□

P5 □□□□□□□□□□□□□□□□ PL_pred < AL □□□□□□ original checkCollisionCallback() □□□□: original safety check □ obstacle / depth / swarm□P5 □ integrity hard safety□

□□□□:

- □□ ARAIM monitor unsafe □□□□□ EMERGENCY_STOP□
- □□□□ B-spline □□□□□□□ predicted PL □□ AL□□□ REPLAN_TRAJ □ EMERGENCY_STOP□
- Predictor □ future AL □□□□□□□ safe□□ conservative branch□

□□ original EGO □□□□

| original □□ | P5 □□□□ |
|---|---|
| EGOReplanFSM::checkCollisionCallback() | □□ collision check □□□ checkTrajIntegrity() |
| EGOPlannerManager::local_data_ | □□□□□□ B-spline□start time□duration |
| changeFSMExecState(REPLAN_TRAJ, ...) | P5 non-imminent violation □□ |
| changeFSMExecState(EMERGENCY_STOP, ...) | P5 current unsafe □ imminent violation □□ |

PROF □□

```
/iap/integrity:
  current certified HPL/VPL/HAL/VAL/IM/state

EGO current trajectory:
  local_data_.position_traj_
  local_data_.start_time_
  local_data_.duration_

Predictor adapter:
  future HPL/VPL at p(t_now + tau)

AlertLimitProvider:
  future HAL/VAL policy at sampled position

FSM parameters:
  p5_horizon_s, p5_dt_s, emergency_time_, safety margin
```

```

IntegrityAction:
    OK
    REPLAN
    EMERGENCY

PlannerIntegrityStatus:
    action
    reason
    current_im_min
    future_min_im
    first_violation_tau_s
    predictor_available
    al_available
    
```

11.1.1

```

struct PlannerIntegrityCheckRequest {
    UniformBspline position_traj;
    double trajectory_start_time_s;
    double trajectory_duration_s;
    double now_s;
};

struct PlannerIntegrityCheckResult {
    IntegrityAction action;
    std::string reason;
    double current_im_min_m;
    double future_min_im_m;
    double first_violation_tau_s;
    bool predictor_available;
    bool al_available;
};

class PlannerIntegritySupervisor {
public:
    PlannerIntegrityCheckResult check(
        const PlannerIntegrityCheckRequest& request) const;
};
    
```

— Mermaid 11.1.1
PROF

```

flowchart TD
    A["P5 tick<br/>called from checkCollisionCallback()"] --> B[" /iap/integrity<br/>current certified monitor"]
    B --> C[" monitor safe?"]
    C -- "IM: state unsafe IM < 0" --> Z[" EMERGENCY<br/>FSM -> EMERGENCY_STOP"]
    C -- "IM" --> D[" local_data_ B-spline<br/>t_cur "]
    D --> E[" p(t_cur + tau)"]
    E --> F["Predictor HPL_pred / VPL_pred"]
    F --> G["Predictor valid?"]
    G -- "IM" --> R[" REPLAN EMERGENCY<br/> tau emergency_time "]
    G -- "IM" --> H["AlertLimitProvider HAL / VAL"]
    H --> I["AL valid?"]
    I -- "IM" --> R
    I -- "IM" --> J[" IM = min(HAL-HPL_pred, VAL-VPL_pred)"]
    J --> K["IM < safety_margin?"]
    K -- "imminent" --> Z
    K -- "imminent" --> Y[" REPLAN<br/>FSM -> REPLAN_TRAJ"]
    K -- "IM" --> L["?"]
    L -- "IM" --> E
    L -- "IM" --> X[" OK<br/> EXEC_TRAJ"]
    
```

- P5 编译器 cost 在 PL/AL/IM
- P5 通过 `/iap/integrity_cost_field` 在 cost 中做硬决策
- v1 中的 FSM state 在 REPLAN_TRAJ 和 EMERGENCY_STOP
- P5 中的 original safety callback

8.3 P2 □□□□□□: candidate ranking

☐ ☐ ☐ ☐

P2 original EGO manager/use_distinctive_trajs

□□□□:

- collision-free $\rightarrow$ optimizer minimizes mean risk $\rightarrow$ min IM $\rightarrow$ collision
- collision-free
- P5 $\rightarrow$ hard safety gate

original EGO

| original | P2 |
|--|--------------------|
| <code>EGOPlannerManager::reboundReplan()</code> | P2 STEP 2 optimize |
| <code>pp_.use_distinctive_trajs</code> | P2 |
| <code>BsplineOptimizer::distinctiveTrajs(segments)</code> | |
| <code>BsplineOptimizeTrajRebound(ctrl_pts_temp, final_cost, trajs[i], ts)</code> | |
| <code>if (final_cost < min_cost)</code> | score |

11

```

optimized candidate control points
candidate final_cost from original optimizer
UniformBspline(candidate_ctrl_pts, order=3, ts)
PlannerRiskField query result
candidate ranking weights:
    lambda_candidate_integrity
    unknown_penalty
    min_im_penalty

```

PROF

```
winning_candidate control points
candidate debug metrics:
  final_cost
  integrity_score
  combined_score
  mean_cost
  min_im
  unknown_ratio
```

[illegible]

```
struct CandidateIntegrityScore {
    double mean_cost;
    double min_im;
```

000

PROF

- obstacle safety & feasibility minimizes risk
- optimizer minimizes cost
- P5 replan/emergency

original EGO

| original | P1 |
|---|---|
| <code>BsplineOptimizer::setParam()</code> | <code>lambda_integrity</code> <code>gradient clip</code> <code>unknown penalty</code> |
| <code>BsplineOptimizer::combineCostRebound()</code> | <code>f_integrity</code> <code>g_integrity</code> |
| <code>rebound_optimize()</code> | L-BFGS cost/gradient |
| <code>calcDistanceCostRebound()</code> | collision cost obstacle clearance P1 |

```
B-spline control points q
PlannerRiskField cost/gradient sample
lambda_integrity
integrity_grad_norm_max
integrity_cost_max
unknown penalty for soft cost
```

```
f_integrity
g_integrity
updated f_combine
updated grad_3D
debug metrics:
  hit_count
  miss_count
  clipped_grad_count
  mean_integrity_cost
```

```
void BsplineOptimizer::calcIntegrityCost(
    const Eigen::MatrixXd& q,
    double& cost,
    Eigen::MatrixXd& gradient);
```

PROF

```
Algorithm P1: Backend soft integrity cost

Input:
  q          B-spline control points
  risk_field PlannerRiskField
  lambda_integrity
  cost_max
  grad_norm_max

Output:
  f_integrity
  g_integrity

Procedure:
  1: f_integrity <- 0
  2: g_integrity <- zero matrix with same shape as q
  3: for each internal optimized control point q_i do
  4:   sample <- risk_field.query(q_i, now)
```

```

5:     if sample is valid then
6:         c <- clamp(sample.cost, 0, cost_max)
7:         g <- sample.grad
8:         if norm(g) > grad_norm_max then
9:             g <- g / norm(g) * grad_norm_max
10:        end if
11:        f_integrity <- f_integrity + c
12:        g_integrity[:, i] <- g_integrity[:, i] + g
13:    else
14:        f_integrity <- f_integrity + unknown_soft_penalty
15:    end if
16: end for
17: f_combine <- f_combine + lambda_integrity * f_integrity
18: grad_3D <- grad_3D + lambda_integrity * g_integrity

```

□□□□□□

- P1 □□□ $PL_pred - AL$ □ hard violation □□
- P1 □□□□□□□□□□ collision distance gradient□
- P1 □□□□□ `combineCostRebound()` □□□ original □□□□□
- □ cost field stale/miss□□□□□ 0 risk□soft cost □□□ small unknown penalty □□□□□□□□□

8.5 P3 □□/□□□□□□: task-level corridor bias

□□□□

P3 □□□□□□□□□□□□□□□□□□□□□□ planner □□□□□□□□□□□□□□□□“□□□”□□□□□□ global reference □ local target □□□□□□□□□□ collision cost□

□□□□:

- □□□□□□□□□□ local target □□□□□□□□□ risk □□□
- □□ P1 □□□□□□□
- □□□ global reference □□□ risk □□□□□□□□□□□□□□□

□□ **original EGO** □□□□

| original □□ | P3 □□□□ |
|---|--|
| <code>EGOPlannerManager::planGlobalTraj()</code> | □□ global min-snap reference □□□□□□□ |
| <code>EGOPlannerManager::planGlobalTrajWaypoints()</code> | waypoint/intermediate point □□ risk-aware □□ |
| <code>EGOReplanFSM::getLocalTarget()</code> | □ global reference □□ local target □□□□□□□□ |
| <code>callReboundReplan()</code> | □□□□□□ local target □□□□ replan |

PROF

□□

start point
 nominal global reference or nominal local target
 planning horizon
 PlannerRiskField risk query
 local occupancy map for basic feasibility check
 bias radius / lateral samples / max target shift

□□

bias-adjusted local target or reference point
 risk metrics before/after bias
 fallback flag if no better feasible target is found

□□□□□□

```
Eigen::Vector3d biasLocalTargetByRisk(  
    const Eigen::Vector3d& start,  
    const Eigen::Vector3d& nominal_target,  
    double horizon_m,  
    const PlannerRiskField& risk_field,  
    const GridMap& grid_map);
```

□□□

Algorithm P3: Risk-biased local target selection

Input:

```
start  
nominal_target from getLocalTarget()  
risk_field  
grid_map  
lateral_offsets
```

Output:

```
selected_target
```

Procedure:

```
1: candidates <- {nominal_target}  
2: construct lateral offset candidates around nominal target  
3: best_score <- score(nominal_target)  
4: selected_target <- nominal_target  
5: for each candidate p do  
6:     if p is outside local map or occupied then  
7:         continue  
8:     end if  
9:     risk <- risk_field.query(p, now)  
10:    if risk unavailable then  
11:        continue  
12:    end if  
13:    score <- distance_to_nominal_weight * norm(p - nominal_target)  
14:           + risk_weight * risk.cost  
15:    if score < best_score then  
16:        best_score <- score  
17:        selected_target <- p  
18:    end if  
19: end for  
20: return selected_target
```

PROF

□□□□□□□

- P3 □□□ local target □□□□□□□□ occupied □□□
- P3 □□ reference bias□□□□ A* □ P5□
- □□□□□□□□□□□□ nominal target□
- P3 □□□ P1/P2 □□□□□□□□□□□□□□□□□□

8.6 P4 Risk-aware A*: front-end guide extension

□□□□

P4 □□□□□ original A* guide path □□□□□□ risk-aware edge cost□□□□□□□□□□ guide □□□□□□□□□□□□□□□□□□□□□□□□ P1/P2/P3 □□□□□□□□□□□□□□

□□□□:

- □□□□□□□□□□□□□□□□ A* □□□□□□□□ risk □□□□ guide path□
- □ rebound optimizer □□□□□□ **base_point** / **direction**□□□□□□□□□□□□□□□□□□ basin□

- original A* occupancy hard rejection

original EGO

| original | P4 |
|--|-----------------------------------|
| BsplineOptimizer::initControlPoints() | collision segment A* |
| AStar::AstarSearch(step_size, in, out) | neighbor expansion risk edge cost |
| checkOccupancy(Index2Coord(neighbor)) | occupied voxel |
| tentative_gScore = current->gScore + static_cost | static_cost + risk_cost |

A* current node and neighbor node
neighbor position
PlannerRiskField or front-search risk field
lambda_integrity_front
unknown policy for front search
original occupancy map

risk-aware A* path
updated gScore/fScore
debug metrics:
 expanded nodes
 mean edge risk
 unknown edge count

```
double AStar::edgeCostWithRisk(
    const Eigen::Vector3d& current_pos,
    const Eigen::Vector3d& neighbor_pos,
    double geometric_cost) const;
```

PROF

Algorithm P4: Risk-aware A* edge expansion

Input:

- current node
- neighbor node
- grid_map occupancy
- risk_field
- lambda_integrity_front

Output:

- updated open set with risk-aware scores

Procedure:

- for each neighbor of current do
- p_neighbor <- Index2Coord(neighbor.index)
- if grid_map.getInflateOccupancy(p_neighbor) then
- continue
- end if
- static_cost <- geometric distance from current to neighbor


```

7:   risk_sample <- risk_field.query(p_neighbor, now)
8:   if risk_sample valid then
9:     edge_cost <- static_cost * (1 + lambda_integrity_front * risk_sample.cost)
10:  else
11:    edge_cost <- static_cost + unknown_front_penalty
12:  end if
13:  tentative_gScore <- current.gScore + edge_cost
14:  update neighbor if tentative_gScore improves previous score
15: end for

```

| | | | | | | |
|--|--|--|--|--|--|--|
| | | | | | | |
|--|--|--|--|--|--|--|

- Occupancy 00 hard rejection0risk 000000
- P4 000 A* 0000 Predictor 000000000000 risk query callback0
- 0 risk field unavailable000000 original A* 000 conservative fallback0
- P4 0000000000 P5/P1/P2 0000000

8.7 P0-P5 □ original □□□□□□

□□□□□□□□:

```
P0 read-only data foundation
-> P5 hard supervisor
-> P2 candidate ranking
-> P1 backend soft cost
-> P3 reference bias
-> P4 risk-aware A* only if needed
```

□□□□□□□:

| | | |
|----|--------------------------|--------------|
| P0 | PL_pred < AL hard safety | PL_pred < AL |
| P5 | PL_pred < AL hard safety | PL_pred < AL |
| P2 | PL_pred < AL hard safety | PL_pred < AL |
| P1 | PL_pred < AL hard safety | PL_pred < AL |
| P3 | PL_pred < AL hard safety | PL_pred < AL |
| P4 | PL_pred < AL hard safety | PL_pred < AL |

PROF

□ □ □ □ □ □ □ □ □ □ □ □ :

```
original EGO safety:
  occupancy / depth / swarm -> replan or emergency

IAP integrity safety:
  ARAIM current monitor + Predictor future PL + AL policy -> replan or emergency
```

EGO planner

9. Safety Planner Mermaid

original EGO planner IAP Safety Planner

```
flowchart TD
    %% =====
    %% Color convention
    %% =====
    classDef original fill:#dbeafe,stroke:#2563eb,stroke-width:1.5px,color:#0f172a;
```

```
classDef safety fill:#ffedd5,stroke:#f97316,stroke-width:1.5px,color:#7c2d12;
```

```
%% =====
```

```
%% Original inputs and map
```

```
%% =====
```

```
OI0["Original 00: odom_world<br/>0000/00/00"] --> OFSM0
```

```
OI1["Original 00: grid_map/odom<br/>grid_map/cloud / depth"] --> OMAP0
```

```
OI2["Original 00: goal / trigger<br/>waypoint 00000"] --> OFSM1
```

```
OI3["Original 00: swarm trajectories<br/>000000 B-spline"] --> OSM0
```

```
OMAP0["GridMap::initMap(<br/>00000/00/00"] --> OMAP1["GridMap update<br/>raycast + log-odds  
occupancy"]
```

```
OMAP1 --> OMAP2["Inflated occupancy<br/>getInflateOccupancy(pos)"]
```

```
%% =====
```

```
%% Safety data foundation
```

```
%% =====
```

```
SI0["Safety 00: /iap/integrity<br/>ARAIM current certified monitor"] --> SP5
```

```
SI1["Safety 00: PredictorModule<br/>future advisory HPL/VPL query"] --> SP0
```

```
SI2["Safety 00: evaluator / risk grid<br/>/iap/integrity_cost_field 00"] --> SP0
```

```
SI3["Safety 00: AlertLimitProvider<br/>cloud_clearance 0 fixed AL"] --> SP5
```

```
SP0["P0 PlannerRiskField / Adapter<br/>0000000 risk / PL / cost / grad"] --> SP2
```

```
SP0 --> SP1
```

```
SP0 --> SP3
```

```
SP0 --> SP4
```

```
SP0 -. "P5 future PL 000000 adapter<br/>000 cost 00 PL/AL/IM" .-> SP5
```

```
%% =====
```

```
%% FSM and target handling
```

```
%% =====
```

```
OFSM0["EG0ReplanFSM::init(<br/>00 manager 0timer 0pub/sub"] --> OMGR0
```

```
OMGR0["EG0PlannerManager::initPlanModules(<br/>0000 GridMap / BsplineOptimizer / A*"] --> OMAP0
```

```
OMGR0 --> OOPT0["BsplineOptimizer::setParam(<br/>00 original optimizer 00"]
```

```
OMGR0 --> OA0["AStar::initGridMap(grid_map)"]
```

```
OMGR0 -. "0000 risk adapter" .-> SP0
```

```
OFSM1["waypointCallback() / triggerCallback()"] --> OG0["planNextWaypoint()"]
```

```
OG0 --> OG1["planGlobalTraj(<br/>00 original global min-snap reference"]
```

```
OG1 -.-> SP3["P3 Reference Bias<br/>0 global reference / local target 0 risk-aware 00"]
```

```
SP3 --> OG2["00 biased 0 original global reference<br/>GlobalTrajData"]
```

```
OG1 --> OG2
```

```
OSW0["swarmTrajsCallback(<br/>0000000000"] --> OSW1["original swarm collision check"]
```

```
%% =====
```

```
%% FSM execution and local replan
```

```
%% =====
```

```
OFSM0 --> OT0["10 ms execFSMCallback(<br/>FSM 000"]
```

```
OT0 --> OS0{"FSM state"}
```

```
OS0 -->|"GEN_NEW_TRAJ / SEQUENTIAL_START"| OP0["planFromGlobalTraj()"]
```

```
OS0 -->|"REPLAN_TRAJ"| OP1["planFromCurrentTraj()"]
```

```
OS0 -->|"EXEC_TRAJ"| OEX0["original 00000<br/>00000 / replan threshold"]
```

```
OS0 -->|"EMERGENCY_STOP"| OEMG0["callEmergencyStop()"]
```

```
OP0 --> OC0["callReboundReplan()"]
```

```
OP1 --> OC0
```

```
OC0 --> OL0["getLocalTarget(<br/>0 global reference 0 local target"]
```

```
OL0 -.-> SP3
```

```
SP3 --> OL1["00 biased local target<br/>00000000 original target"]
```

```
OL0 --> OL1
```

```
OL1 --> ORR0["reboundReplan(<br/>local B-spline planning"]
```

```
%% =====
```

```
%% Initial B-spline and A*
```

```
%% =====
```

```
ORR0 --> OINIT0["STEP 1 INIT<br/>polynomial 0 current-traj continuation"]
```

```
OINIT0 --> OBS0["UniformBspline::parameterizeToBspline(<br/>00 ctrl_pts"]
```

```
OBS0 --> OIC0["initControlPoints(ctrl_pts)<br/>0 inflated occupancy 0 collision segments"]
```

```
OMAP2 --> OIC0
```

```

OIC0 --> OA1{"collision segments?"}
OA1 -->|" "| 0OPT1["  rebound optimization"]
OA1 -->|" "| OA2["Original AStar::AstarSearch()<br/>  guide path"]
SP4["P4 Risk-aware A*<br/>  A* edge cost  risk cost"] --> OA2
SP4 --> OA3["risk-aware guide path<br/>  occupied voxel"]
OA2 --> OA3
OA3 --> OIC1["  base_point / direction<br/>  rebound distance cost  "]
OIC1 --> 0OPT1

%% =====
%% Optimization, P2 and P1
%% =====
0OPT1 --> OD0{"use_distinctive_trajs?"}
OD0 -->|"false"| OB0["  BsplineOptimizeTrajRebound()"]
OD0 -->|"true"| OD1["distinctiveTrajs(segments)<br/>  control points"]
OD1 --> OD2["  rebound optimize<br/>  final_cost  B-spline"]
OD2 --> SP2["P2 Candidate Ranking<br/>  optimizer cost + trajectory integrity score  "]
SP2 --> OB1["winning candidate ctrl_pts"]
OB0 --> OB1

OB1 --> OLB0["rebound_optimize()<br/>L-BFGS  "]
OLB0 --> OCOST0["combineCostRebound()<br/>original cost: smoothness / collision / feasibility /
swarm / terminal"]
SP1["P1 Integrity Soft Cost<br/>calcIntegrityCost(q): cost + clipped gradient"] --> OCOST0
OCOST0 --> OLB1["  original collision check / restart<br/>  refine time"]
OLB1 --> OUP0["updateTrajInfo()<br/>  local_data_ B-spline"]

%% =====
%% Publish and execute
%% =====
OUP0 --> OPUB0["  planning/bspline<br/>publishSwarmTrajs() / visualization"]
OPUB0 --> OTS0["traj_server::bsplineCallback()<br/>  knots"]
OTS0 --> OTS1["traj_server::cmdCallback()<br/>  pos/vel/acc/yaw"]
OTS1 --> OOUT0["  PositionCommand<br/>  "]

%% =====
%% Runtime safety loops
%% =====
OFSM0 --> OSAFE0["50 ms checkCollisionCallback()<br/>original runtime safety"]
OSAFE0 --> OSAFE1{"Depth timeout / occupancy collision / swarm unsafe?"}
OSAFE1 -->|" "| OSAFE_OK["original safety OK"]
OSAFE1 -->|" "| OSAFE_REPLAN["  planFromCurrentTraj()<br/>  REPLAN  EMERGENCY"]

OUP0 --> SP5
SP5["P5 Runtime Integrity Supervisor<br/>current monitor + future trajectory hard check"] -->
SP5A["  /iap/integrity<br/>HPL/VPL/HAL/VAL/IM/state"]
SP5 --> SP5B["  local_data_ B-spline  <br/>Predictor  PL_pred  AL provider  AL"]
SP5A --> SP5C{"Integrity safe?"}
SP5B --> SP5C
SP5C -->|"OK"| OSAFE_OK
SP5C -->|"future violation, not imminent"| SREPLAN["Safety  REPLAN<br/>FSM -> REPLAN_TRAJ"]
SP5C -->|"current unsafe  imminent"| SEMG["Safety  EMERGENCY<br/>FSM -> EMERGENCY_STOP"]

OSAFE_REPLAN --> OT0
SREPLAN --> OT0
SEMG --> OEMG0

%% =====
%% Classes
%% =====
class
OI0,OI1,OI2,OI3,OFSM0,OMGR0,OMAP0,OMAP1,OMAP2,0OPT0,OA0,OFSM1,OG0,OG1,OG2,OSW0,OSW1,OT0,OS0,OP0,OP
1,OEX0,OEMG0,OC0,OL0,OL1,ORR0,OINIT0,OBS0,OIC0,OA1,0OPT1,OA2,OA3,OIC1,OD0,OB0,OD1,OD2,OB1,OLB0,OCO
ST0,OLB1,OUP0,OPUB0,OTS0,OTS1,OOUT0,OSAFE0,OSAFE1,OSAFE_OK,OSAFE_REPLAN original;
class SI0,SI1,SI2,SI3,SP0,SP1,SP2,SP3,SP4,SP5,SP5A,SP5B,SP5C,SREPLAN,SEMG safety;

```

00000000000000:

| 名前 | 説明 | 備考 |
|----|------------------------------|---|
| P0 | PlannerRiskField / Adapter | risk / PL / cost / grad EGO の Predictor/evaluator |
| P5 | Runtime Integrity Supervisor | monitor 監視 PL_pred < AL 監視 |
| P2 | Candidate Ranking | 候補のランク付け |
| P1 | Integrity Soft Cost | L-BFGS 最適化 |
| P3 | Reference Bias | global/local reference 参照 |
| P4 | Risk-aware A* | リスク aware guide path A* 最適化 risk |

安全に関する:

```
Original EGO safety:
depth / occupancy / swarm unsafe -> replan or emergency

IAP Safety Planner:
current ARAIM unsafe 且 future PL_pred >= AL -> replan or emergency
```

10. P0 RiskGridMap と EGO GridMap

P0 は planner-side の P0 は FuturePLFieldPredictor, PLGrid, UnifiedRiskGrid, PIRiskAdapter によって EGO planner の RiskGridMap

概要:

```
separate buffer, same world frame
```

RiskGridMap は EGO GridMap と indexing buffer EGO の occupancy map

10.1 P0 の構成

planner-side の構成:

1. 予測
 - predictor query, grid cache, AL/IM, PI cost, gradient, PointCloud2 schema, stale/unknown policy
 - planner から predictor へ送られる
2. EGO planner の処理
 - EGO planner は GridMap の occupancy map から getInflateOccupancy(pos)
 - Risk field は localization/integrity-side scalar field によって生成される
 - risk 値は EGO occupancy buffer から EGO の map へ送られる

P0 は planner-side の:

```
alert_limit_model.hpp
future_pl_field_predictor.hpp
future_pl_query_result.hpp
integrity_snapshot.hpp
pi_cost_adapter.hpp
pl_grid.hpp
predicted_araim.hpp
unified_risk_grid.hpp
```

P0 の:

```
PredictorModule
EGO の odom / pose
```

```
EGO GridMap  frame / local window / occupancy  planner risk
```

P0 “A* ”:

optimizer / candidate scorer / reference bias / optional A* risk lookup table

P1 L-BFGS $c_{pi}(p)$ $\text{grad } c_{pi}(p)$ Predictor 7 predictor query
P0 predictor risk field

10.2 RiskGridMap EGO GridMap

RiskGridMap EGO GridMap occupancy buffer EGO GridMap

| | EGO GridMap | RiskGridMap |
|----------------|--|--|
| buffer | occupancy_buffer_occupancy_buffer_inflate_ | risk_buffer_ |
| | occupied / free / unknown | localization risk / predicted PL / valid / stale |
| frame | grid_map/frame_id world/map | |
| resolution | 0.1 m | 0.5-1.0 m |
| update rate | odom/cloud/depth | predictor refresh 1-5 Hz |
| rolling window | | UAV risk window |
| | getInflateOccupancy(pos) | queryRisk(pos) getRiskCost(pos) getRiskGrad(pos) |

EGO GridMap :

```
map_origin
map_size
map_min_boundary / map_max_boundary
resolution / resolution_inv
posToIndex()
indexToPos()
toAddress()
boundIndex()
isInMap()
```

PROF

:

- occupancy_buffer_
- getInflateOccupancy()
- EGO GridMap Predictor
- RiskGridMap EGO GridMap frame occupancy rolling window occupied voxel

10.3

```
struct RiskGridMapParams {
    std::string frame_id = "world";

    double resolution_m = 0.75;
    double size_x_m = 30.0;
    double size_y_m = 30.0;
    double size_z_m = 6.0;

    double refresh_period_s = 0.5;
```

```

double stale_timeout_s = 1.0;
double unknown_cost = 10.0;
double cost_max = 100.0;

bool skip_occupied_voxels = true;
bool use_predictor_batch_query = true;
};

struct RiskVoxel {
    double c_pi = NaN;

    double hpl_pred = NaN;
    double vpl_pred = NaN;
    double pl_scalar = NaN;

    double stamp_s = NaN;
    double age_s = NaN;

    bool valid = false;
    bool stale = true;
    bool unknown = true;

    uint32_t source_flags = 0;
};

struct RiskQueryResult {
    bool ok = false;
    bool valid = false;
    bool stale = true;
    bool unknown = true;

    Eigen::Vector3d position;
    double c_pi = NaN;
    Eigen::Vector3d grad_c_pi = Eigen::Vector3d::Zero();

    double hpl_pred = NaN;
    double vpl_pred = NaN;
    double pl_scalar = NaN;
    double age_s = NaN;

    std::string reason;
};

class RiskGridMap {
public:
    void reset(const Eigen::Vector3d& center, const RiskGridMapParams& params);
    void updateFromPredictor(const Eigen::Vector3d& uav_position,
                            double now_s,
                            const EgoGridMapReadOnlyView& ego_grid,
                            PredictorModule& predictor);

    bool queryRisk(const Eigen::Vector3d& p_w,
                  double now_s,
                  RiskQueryResult* out) const;
    bool getRiskCost(const Eigen::Vector3d& p_w,
                    double now_s,
                    double* cost) const;
    bool getRiskGrad(const Eigen::Vector3d& p_w,
                    double now_s,
                    Eigen::Vector3d* grad) const;

private:
    bool posToIndex(const Eigen::Vector3d& p_w, Eigen::Vector3i* id) const;
    Eigen::Vector3d indexToPos(const Eigen::Vector3i& id) const;
    int toAddress(const Eigen::Vector3i& id) const;
    bool isInMap(const Eigen::Vector3d& p_w) const;
};

```

□□□□:

- `RiskVoxel::c_pi` □ planner preference cost□□□□□□
- `hpl_pred/vpl_pred` □ raw predicted PL□□□ P5 □□□□ P5 □□□ `c_pi` □□□□□
- `grad_c_pi` □□□□ voxel □□□□□□ trilinear interpolation □□□□□□□□□□
- miss/stale/unknown □□□□□□□□□□□ 0 risk□

10.4 □□□□

`RiskGridMap` □□□□□□□□ EGO occupancy map□□□□□□:

```
RiskGridMap refresh tick:
1. □□□□ UAV position / odom stamp
2. □ UAV position □ rolling window center
3. □□□□□□□□□□□□□□ risk buffer
4. □□ risk voxel center
5. □□: □□□ EGO GridMap □□ occupied/out-of-range□□□□ occupied/unknown □□□ predictor
6. □□□□□□ PredictorModule □□ hpl_pred/vpl_pred/pl_scalar
7. □□ planner policy □□ c_pi
8. □□ RiskVoxel: c_pi□PL□stamp□age□valid/stale/unknown/source_flags
9. □□ update latency□valid ratio□unknown ratio
```

□□□□:

```
Algorithm P0-Update: Build rolling RiskGridMap

Input:
  uav_position
  now_s
  ego_grid_readonly
  predictor
  risk_grid_params

Output:
  updated RiskGridMap

Procedure:
1: center <- alignRollingCenter(uav_position)
2: if center moved enough then
3:   reset risk grid window around center
4: end if
5: query_batch <- empty
6: for each voxel index id do
7:   p <- indexToPos(id)
8:   if p outside EGO map or occupied and skip_occupied_voxels then
9:     mark voxel unknown/occupied
10:    continue
11:   end if
12:   append p to query_batch
13: end for
14: results <- PredictorModule.batchQuery(query_batch, now_s)
15: for each result do
16:   if result valid and finite then
17:     voxel.hpl_pred <- result.hpl
18:     voxel.vpl_pred <- result.vpl
19:     voxel.c_pi <- computePlannerCost(result)
20:     voxel.valid <- true
21:     voxel.unknown <- false
22:     voxel.stamp_s <- now_s
23:   else
24:     voxel.valid <- false
25:     voxel.unknown <- true
26:     voxel.c_pi <- unknown_cost
27:   end if
28: end for
```

`computePlannerCost(result)` planner policy Predictor v1 PL-only preference:

```
c_pi = clamp(w_h * normalize(hpl_pred) + w_v * normalize(vpl_pred), 0, cost_max)
```

P0 v1 `alert_limit_model` AL/IM policy AL/IM hard safety P5

10.5

P1/P2/P3/P4 cost field:

```
queryRisk(p) -> c_pi + grad_c_pi + valid/stale/unknown
```

Steps:

1. `p_w` risk grid index
2. 8 voxel
3. voxel invalid/stale policy unknown
4. `c_pi` trilinear interpolation
5. trilinear interpolation `grad_c_pi`

Algorithm:

Algorithm P0-Query: Trilinear risk interpolation

Input:

`p_w`
`now_s`
`RiskGridMap`

Output:

`RiskQueryResult`

Procedure:

- 1: if `p_w` outside risk grid then
- 2: return `unknown(out_of_range)`
- 3: end if
- 4: find base cell index and interpolation weights
- 5: gather 8 neighbor voxels
- 6: if required voxels are invalid or stale then
- 7: return `unknown(stale_or_invalid)`
- 8: end if
- 9: `c_pi` <- trilinear(`corner.c_pi`, weights)
- 10: `grad_c_pi` <- `analyticGradientOfTrilinear`(`corner.c_pi`, weights, resolution)
- 11: `hpl_pred` <- trilinear(`corner.hpl_pred`, weights) if requested
- 12: `vpl_pred` <- trilinear(`corner.vpl_pred`, weights) if requested
- 13: return valid result

Implementation:

- L-BFGS P1 cost/gradient
- Predictor `control_points * iterations * 7` query
- grid trilinear value analytic gradient optimizer

unknown policy:

- P1 soft cost: unknown penalty
- P2/P3/P4: unknown penalty
- P5: unknown future PL safe P5 conservative branch

10.6 P1-P5

PROF

P0

| 模块 | 模块 P0 数据 | 数据源 | 数据流 | 数据流 |
|-------------------|---|-------------------------------|----------------------------|-------------------------------------|
| P1 模块 soft cost | c_pi, grad_c_pi, valid/stale | calcIntegrityCost(q) | B-spline 模块 risk 模块 | 模块 c_pi 模块 |
| P2 模块 | 模块 c_pi, unknown ratio, min PL | candidate score | 模块 | 模块 |
| P3 reference bias | target/reference 模块 c_pi | biased local/global reference | 模块 | 模块 occupied/out-of-range 模块 |
| P4 risk-aware A* | cell/edge c_pi | edge risk cost | A* guide path 模块 risk | 模块 risk 模块 occupancy hard rejection |
| P5 supervisor | raw hpl_pred/vpl_pred, valid/stale, timestamp | future PL sample | PL_pred < AL hard check 模块 | 模块 c_pi 模块 PL_pred < AL |

模块 adapter 模块:

```
RiskGridMap
-> RiskCostAdapter for P1/P2/P3/P4
-> PredictedPLAdapter for P5
```

模块:

- RiskCostAdapter 模块 c_pi/grad/unknown
- PredictedPLAdapter 模块 hpl_pred/vpl_pred/valid/stale
- 模块 buffer 模块

10.7 Mermaid 模块

模块 original EGO GridMap 模块 P0 RiskGridMap 模块

```
flowchart TD
    classDef original fill:#dbeafe,stroke:#2563eb,stroke-width:1.5px,color:#0f172a;
    classDef safety fill:#ffedd5,stroke:#f97316,stroke-width:1.5px,color:#7c2d12;

    00["EGO odom / cloud / depth"] --> 01["EGO GridMap<br/>occupancy_buffer<br/>occupancy_buffer_inflate_"]
    01 --> 02["getInflateOccupancy(pos)<br/>obstacle hard safety"]
    02 --> 03["Original A* / rebound optimizer / collision check"]

    S0["Current UAV pose<br/>same world/map frame"] --> S2
    S1["New PredictorModule<br/>batch query predicted PL"] --> S2["P0 RiskGridMap refresh"]
    01 -. "read-only frame / local window / optional occupancy skip" .-> S2
    S2 --> S3["RiskGridMap independent buffer<br/>coarser rolling local grid"]
    S3 --> S4["queryRisk(pos)<br/>trilinear c_pi + analytic grad"]
    S3 --> S5["queryPredictedPL(pos)<br/>hpl_pred / vpl_pred / valid / stale"]

    S4 --> P1["P1 backend soft cost<br/>calcIntegrityCost()"]
    S4 --> P2["P2 candidate ranking"]
    S4 --> P3["P3 reference bias"]
    S4 --> P4["P4 optional risk-aware A* edge cost"]
    S5 --> P5["P5 runtime supervisor<br/>PL_pred + AL -> hard decision"]

    03 --> OUT["Original trajectory output"]
    P1 --> OUT
    P2 --> OUT
    P3 --> OUT
    P4 --> OUT
    P5 --> ACT["REPLAN_TRAJ / EMERGENCY_STOP"]

    class 00,01,02,03,OUT original;
    class S0,S1,S2,S3,S4,S5,P1,P2,P3,P4,P5,ACT safety;
```

□□□□□□□□:

- EGO GridMap $\rightarrow$ RiskGridMap $\rightarrow$ $\rightarrow$ $\rightarrow$ $\rightarrow$ $\rightarrow$ map
- $\rightarrow$ $\rightarrow$ frame $\rightarrow$ $\rightarrow$ buffer $\rightarrow$ $\rightarrow$ $\rightarrow$ $\rightarrow$ $\rightarrow$
- EGO GridMap $\rightarrow$ $\rightarrow$ $\rightarrow$ obstacle hard safety
- RiskGridMap $\rightarrow$ $\rightarrow$ integrity preference lookup $\rightarrow$ future PL cache
- P5 $\rightarrow$ hard decision $\rightarrow$ $\rightarrow$ supervisor $\rightarrow$ $\rightarrow$ $\rightarrow$

10.8 □□□□

□□□□:

- `planner-side` headers:

FuturePLFieldPredictor / FuturePLQueryResult / IntegritySnapshot
PLGrid / UnifiedRiskGrid / PICostAdapter / PredictedAraim / alert_limit_model

- risk grid $\rightarrow$ EGO GridMap $\rightarrow$ occupancy buffer
- RiskGridMap $\rightarrow$ A* $\rightarrow$ hard obstacle rejection $\rightarrow$ EGO GridMap
- c_{pi} $\rightarrow$ P5 hard safety statement
- Predictor $\rightarrow$ planner grid extent $\rightarrow$ voxel index $\rightarrow$ PointCloud2 schema $\rightarrow$ EGO optimizer
- original EGO planner

□□□□□□:

```

1. 00 RiskGridMap 0000000000
2. 00 PredictorModule batch query000 read-only refresh
3. P1 00 getRiskCost/getRiskGrad
4. P2/P3 00 queryRisk 0 scoring/bias
5. P4 0000 risk edge cost
6. P5 000 raw predicted PL/valid/stale0000 c_pi 0000

```

□□□□□□:

- `PlannerRiskField` `RiskGridMap` + adapter `UnifiedRiskGrid`
- `UnifiedRiskGrid` `PLGrid` `PICostAdapter` `RiskGridMap`
- `P0` `RiskGridMap`

PROF 11. P5 Forward Recovery Runtime Integrity Supervisor

PROF

```
P5 "current unsafe EMERGENCY_STOP" P5 P5  
EGO EmergencyStop()
```

11.1 □□□□□□

P5 □□□□□□□□:

1. ☐ certified monitor ☐ unsafe ☐
2. ☐ unsafe ☐
3. ☐ unsafe ☐

□□ P5 □□□□□□:

- [illegible]

11.2 □□□□□□□□

P5 Runtime Integrity Supervisor 模块:

| 模块 | 名称 | 功能 |
|-------|--------------------------------|--|
| 完整性监测 | /iap/integrity | 监测 hpl/vpl/hal/val/im_min/integrity_state 是否 certified monitor 是否 safe |
| 规划器 | EGOPlannerManager::local_data_ | 存储 B-spline 规划数据 |
| 预测器 | P0 RiskGridMap 和 Predictor | 预测 HPL/VPL |
| 警报限制 | P5AlertLimitPolicy | 计算 HAL/VAL 安全裕度 clearance |
| 网格图 | EGO GridMap | 环境网格图 |
| 任务管理器 | EGO FSM / manager | 管理 forward recovery 和 mission replan |
| 回溯恢复 | P5 模块 | 实现 backtrack recovery |

P5 变量:

```
last_safe_pose
last_safe_time
last_safe_im_min
last_safe_velocity
last_p5_action
last_violation_reason
```

在 /iap/integrity 模块 safe 变量 im_min 存储 last_safe_pose 和 last_safe_pose 的 backtrack 数据

P5 动作:

| P5 action | 描述 | EGO FSM 动作 |
|---------------------------|---------------------------------|---------------|
| OK | 正常状态 | EXEC_TRAJ |
| REPLAN_MISSION | 安全重新规划 mission/local target | REPLAN_TRAJ |
| FORWARD_RECOVERY_REPLAN | 不安全重新规划 forward recovery target | REPLAN_TRAJ |
| LATERAL_RECOVERY_REPLAN | 侧向重新规划 lateral recovery target | REPLAN_TRAJ |
| BACKTRACK_RECOVERY_REPLAN | 回溯重新规划 last_safe_pose | REPLAN_TRAJ |
| EMERGENCY_STOP | 紧急停止 stop_pos | EmergencyStop |

11.3 future unsafe 预测

P5 模块“未来不安全”预测器 P2 和 P5 模块 final gate 预测器

预测 B-spline:

```
t_cur = now - local_data_.start_time
t_end = min(local_data_.duration, t_cur + p5_horizon_s)
dt = p5_sample_dt
```

tau 在 0 到 t_end - t_cur 之间:

```
p_tau = trajectory.evaluate(t_cur + tau)
HPL_pred, VPL_pred = RiskGridMap.queryPredictedPL(p_tau, now + tau)
HAL, VAL = P5AlertLimitPolicy.evaluate(p_tau, now + tau)
```

```
im_h = HAL - HPL_pred
im_v = VAL - VPL_pred
im_min = min(im_h, im_v)
```

unsafe 条件:

```
sample_unsafe =
    predictor_unavailable
    OR alert_limit_unavailable
    OR risk_sample_stale
    OR im_min < im_margin_min
```

predictor_unavailable / alert_limit_unavailable / risk_sample_stale 保守未知
unknown 已确认 unsafe 硬关卡

unsafe 条件“unsafe”:

```
future_unsafe =
    first_unsafe_tau exists AND (
        first_unsafe_tau <= emergency_time_s
        OR consecutive_unsafe_count >= min_consecutive_unsafe
        OR unsafe_ratio >= max_unsafe_ratio
        OR min_im < severe_negative_im_threshold
    )
```

条件:

- first_unsafe_tau <= emergency_time_s 紧急停止
- consecutive_unsafe_count >= min_consecutive_unsafe
- unsafe_ratio >= max_unsafe_ratio horizon
- min_im < severe_negative_im_threshold

unsafe sample debug/status safety tick REPLAN_MISSION

11.4 current unsafe

/iap/integrity unsafe P5 “”

条件:

- Forward recovery** local target global reference predicted PL IM
- Lateral recovery** horizon
- Backtrack recovery** last_safe_pose occupancy-free
- Emergency stop** occupancy predicted PL AL EmergencyStop()

Forward recovery 条件:

- planner waypoint
- unsafe EGO planner
- P5 future check backtrack

Backtrack recovery 条件:

```
use_backtrack =
    forward_recovery_failed
    AND lateral_recovery_failed
    AND last_safe_pose_valid
    AND path_to_last_safe_pose_collision_free
    AND predicted_integrity_to_last_safe_pose_acceptable
```

11.5 original EGO FSM

P5 original EGO FSM `checkCollisionCallback()` safety tick P5 action

| FSM | P5 action | |
|----------------|--|--|
| EXEC_TRAJ | OK | planning/bspline |
| EXEC_TRAJ | REPLAN_MISSION | REPLAN_TRAJ planFromCurrentTraj() local target |
| EXEC_TRAJ | FORWARD_RECOVERY_REPLAN | REPLAN_TRAJ manager forward recovery target |
| EXEC_TRAJ | LATERAL_RECOVERY_REPLAN | REPLAN_TRAJ manager lateral recovery target |
| EXEC_TRAJ | BACKTRACK_RECOVERY_REPLAN | REPLAN_TRAJ manager backtrack target |
| | EMERGENCY_STOP | callEmergencyStop(odom_pos_) EMERGENCY_STOP |
| EMERGENCY_STOP | recovery target found and velocity small | stop trajectory GEN_NEW_TRAJ recovery replan |

EMERGENCY_STOP B-spline P5 recovery replan EmergencyStop()

11.6 Mermaid

P5 original EGO planner

```
graph TD
    A["EGO FSM<br/>EXEC_TRAJ"] --> B["Original checkCollisionCallback()<br/>obstacle / swarm / depth checks"]
    B --> C["Current executing B-spline<br/>local_data_"]
    C --> P0["P5 current monitor check"]
    P0 --> P1["Sample executing trajectory<br/>t_cur to min(duration, t_cur + horizon)"]
    P1 --> P2["Future PL query"]
    P2 --> P3["Future AL query"]
    P3 --> P4["Compute future IM<br/>im_min = min(HAL-HPL, VAL-VPL)"]
    P4 --> P5["Classify future unsafe<br/>first tau / consecutive / ratio / severity"]
    P5 --> D0["Current safe?"]
    P5 --> D1["Future acceptable?"]
    D0 -- "yes" --> D1
    D1 -- "yes" --> OK["P5 action: OK<br/>keep EXEC_TRAJ"]
    D1 -- "non-imminent unsafe" --> RM["P5 action: REPLAN_MISSION<br/>same mission target"]
    D1 -- "unsafe" --> FR
    D1 -- "imminent or severe unsafe" --> FR["Search forward recovery target<br/>mission direction / local target corridor"]
    FR --> D2["Forward target valid?"]
    D2 -- "yes" --> AFR["P5 action: FORWARD_RECOVERY_REPLAN"]
    D2 -- "no" --> LR["Search lateral recovery target"]
    LR --> D3["Lateral target valid?"]
    D3 -- "yes" --> ALR["P5 action: LATERAL_RECOVERY_REPLAN"]
    D3 -- "no" --> BR["Check last_safe_pose<br/>and path back"]
    BR --> D4["Backtrack valid?"]
    D4 -- "yes" --> ABR["P5 action: BACKTRACK_RECOVERY_REPLAN"]
    D4 -- "no" --> ES["P5 action: EMERGENCY_STOP"]
```

```

OK --> E0["FSM remains EXEC_TRAJ"]
RM --> E1["FSM -> REPLAN_TRAJ<br/>planFromCurrentTraj()"]
AFR --> E2["FSM -> REPLAN_TRAJ<br/>temporary forward target"]
ALR --> E3["FSM -> REPLAN_TRAJ<br/>temporary lateral target"]
ABR --> E4["FSM -> REPLAN_TRAJ<br/>temporary backtrack target"]
ES --> E5["FSM -> EMERGENCY_STOP<br/>EmergencyStop(odom_pos_)"]

class A,B,C,E0,E1,E2,E3,E4,E5 original;
class I,R,L,P0,P1,P2,P3,P4,P5,D0,D1,FR,D2,AFR,LR,D3,ALR,BR,D4,ABR,RM,OK safety;
class ES danger;

```

11.7 □□□□□□□□

Algorithm: Runtime Integrity Supervisor with Forward Recovery

Input:

```

current_integrity from /iap/integrity
executing trajectory T_exec
current trajectory time t_cur
RiskGridMap or Predictor interface Q_PL
P5AlertLimitPolicy Q_AL
EGO GridMap occupancy query Q_occ
mission target and local target
last_safe_pose memory

```

Output:

```

P5Action action
optional recovery_target
planner_integrity_status

```

```

1: if current_integrity is fresh and current_integrity.im_min >= safe_update_margin then
2:   update last_safe_pose with current odom pose
3: end if

4: current_safe <- isCurrentMonitorSafe(current_integrity)

5: future_stats <- empty
6: for tau from 0 to min(T_exec.duration - t_cur, p5_horizon_s) step p5_sample_dt do
7:   p <- T_exec.evaluate(t_cur + tau)
8:   pl <- Q_PL.queryPredictedPL(p, now + tau)
9:   al <- Q_AL.evaluate(p, now + tau)
10:  if pl unavailable or al unavailable or pl stale then
11:    mark sample as UNKNOWN_UNSAFE
12:  else
13:    im_h <- al.HAL - pl.HPL_pred
14:    im_v <- al.VAL - pl.VPL_pred
15:    im_min <- min(im_h, im_v)
16:    mark sample unsafe if im_min < im_margin_min
17:  end if
18:  update future_stats with sample state, tau, and im_min
19: end for

20: future_unsafe <- classifyFutureUnsafe(
21:   future_stats.first_unsafe_tau,
22:   future_stats.consecutive_unsafe_count,
23:   future_stats.unsafe_ratio,
24:   future_stats.min_im)

25: if current_safe and not future_unsafe then
26:   return OK
27: end if

28: if current_safe and future_unsafe is non-imminent then
29:   return REPLAN_MISSION
30: end if

31: forward_target <- searchForwardRecoveryTarget(

```

```
32:     current_pose, local_target, mission_reference, Q_PL, Q_AL, Q_occ)
33: if forward_target is valid then
34:     return FORWARD_RECOVERY_REPLAN with forward_target
35: end if

36: lateral_target <- searchLateralRecoveryTarget(
37:     current_pose, allowed_radius, allowed_height_band, Q_PL, Q_AL, Q_occ)
38: if lateral_target is valid then
39:     return LATERAL_RECOVERY_REPLAN with lateral_target
40: end if

41: if last_safe_pose is valid then
42:     backtrack_target <- validateBacktrackTarget(last_safe_pose, Q_PL, Q_AL, Q_occ)
43:     if backtrack_target is valid then
44:         return BACKTRACK_RECOVERY_REPLAN with backtrack_target
45:     end if
46: end if

47: return EMERGENCY_STOP
```

11.8

| Predictor unavailable | safe conservative unknown |
|-----------------------------------|--------------------------------|
| RiskGridMap sample stale | 0 risk unknown unsafe |
| AL unavailable | safe |
| unsafe sample | debug/status |
| unsafe sample | REPLAN_MISSION recovery replan |
| future violation emergency_time_s | EMERGENCY_STOP |
| current unsafe | FORWARD_RECOVERY_REPLAN |
| current unsafe | lateral backtrack |
| last_safe_pose | backtrack |
| recovery target | |
| P5 | original EGO planner |

11.9

PROF

P5 original safety check optimizer cost:

- checkCollisionCallback() obstacle / swarm / depth
 - checkTrajIntegrity() PlannerIntegritySupervisor::check() P5 action
 - REPLAN_MISSION planFromCurrentTraj()
 - recovery replan planner manager “” waypoint
 - EMERGENCY_STOP
 - /iap/planner_integrity_status
action/current_im_min/future_min_im/first_unsafe_tau/reason/recovery_target_valid
- “”: P5 “” hard safety supervisor PL_pred >= AL